



DOCUMENTACIÓN TÉCNICA

Configuración Gateway DUSUN – Infraestructura LoRaWAN con ChirpStack y Home Assistant

Autor: José L. Márquez Montoro

Empresa: Synergia Tech

Fecha: 11/02/2026

Contents

1. Introducción.....	3
2. Requisitos Previos.....	3
3. Arquitectura del Sistema.....	3
4. Instalación del Sistema Operativo.....	3
5. Configuración Inicial del Sistema.....	4
6. Configuración Gateway LoRaWAN.....	4
7. Configuración MQTT (Mosquitto).....	4
8. Instalación y Configuración de ChirpStack.....	4
9. Integración con Home Assistant.....	5
10. Acceso Remoto Seguro – Tailscale.....	5
11. Buenas Prácticas y Mantenimiento.....	5
12. Conclusión.....	5
13. Anexo Técnico – Comandos de Configuración.....	5
1.1 Instalador de DUSUN.....	6
1.2 SERVICIO MOSQUITTO.....	7
2. CONFIGURAR CHIRPSTACK.....	8
2.1 Javascript AXIOMA W1:.....	8
2.2 Javascript CONTADORES CHINOS:.....	13
3. COMANDO CLAVE DESPUES DE HABER PUESTO LAS IPS:.....	19
4. CONFIGURAR WIFI.....	19
5. CONFIGURACION HOME ASSISTANT:.....	20

1. Introducción

Este documento describe el procedimiento completo para la configuración de una máquina DUSUN como Gateway LoRaWAN, incluyendo la instalación del sistema operativo, configuración del Packet Forwarder, despliegue de ChirpStack mediante Docker, integración con MQTT, configuración de Home Assistant y acceso remoto seguro mediante Tail Scale.

El objetivo es desplegar una infraestructura IoT escalable que permita la recepción y gestión de datos desde dispositivos LoRaWAN (contadores de agua), garantizando modularidad, seguridad y mantenibilidad.

2. Requisitos Previos

Hardware:

- Máquina DUSUN compatible con LoRaWAN
- Antena LoRa EU868
- Conectividad Ethernet o WiFi

Software:

- Android Tool
- Imagen del sistema operativo DUSUN
- Cliente SSH
- Docker y Docker Compose
- Cuenta en Tailscale

Red:

- Acceso a red local
- Acceso a Internet

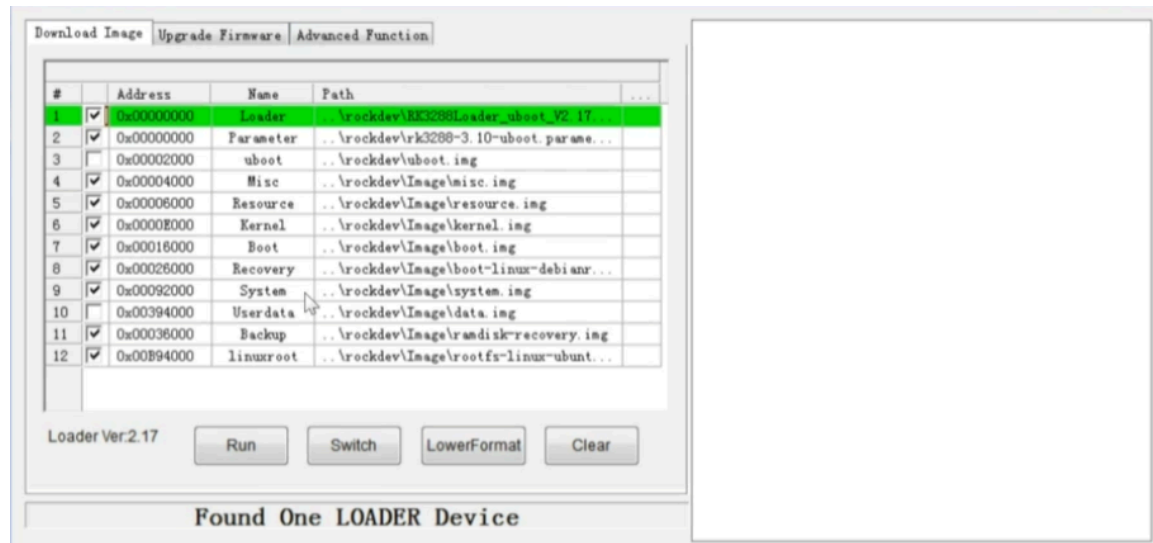
3. Arquitectura del Sistema

Dispositivos LoRaWAN → Gateway DUSUN → ChirpStack → Mosquitto (MQTT) → Home Assistant → Tailscale

La arquitectura es modular, escalable, basada en estándares abiertos y preparada para despliegues multi-dispositivo.

4. Instalación del Sistema Operativo

1. Conectar la máquina al PC mediante USB.
2. Abrir Android Tool.
3. Activar modo Loader.



4. Cargar la imagen del sistema.
5. Ejecutar el flasheo y verificar instalación correcta.

Se recomienda validar la integridad del sistema tras la instalación.

6. Tras el flasheo, se utilizó un IP Scanner para identificar la dirección IP asignada al dispositivo dentro de la red local.

Advanced IP Scanner

Archivo Vista Configuración Ayuda

Explorar

192.168.1.1-192.168.1.254

Lista de resultados Favoritos

Estado	Nombre	IP	Fabricante	Dirección MAC	Comentarios
>	192.168.1.1	192.168.1.1	REALTEK SE...	00:E0:4C:98:14:FB	
	192.168.1.100	192.168.1.100		B6:9F:92:15:20:94	
	LAPTOP-QBVUV...	192.168.1.101	CLOUD NET...	38:D5:7A:24:7F:3F	
	192.168.1.102	192.168.1.102		F2:B3:F2:BD:16:A4	
	192.168.1.104	192.168.1.104		D6:F3:FA:F8:F6:00	
	192.168.1.105	192.168.1.105		08:9D:F4:6C:90:17	
	192.168.1.107	192.168.1.107		B2:F1:5F:98:37:2D	
	HP17CW5NS	192.168.1.108		E4:60:17:7D:F4:79	
	HomeAssistant	192.168.1.116	Deqing Dusu...	30:AE:7B:EB:40:C1	
	WIN-R5U4V3S0...	192.168.1.117		C0:BF:BE:07:C4:2B	
	HomeAssistant	192.168.1.118	Deqing Dusu...	30:AE:7B:EB:40:37	
	HomeAssistant	192.168.1.119	Deqing Dusu...	30:AE:7B:EB:9A:42	
	HomeAssistant	192.168.1.120	Deqing Dusu...	30:AE:7B:EB:40:00	
	HomeAssistant	192.168.1.121	Deqing Dusu...	30:AE:7B:EB:40:BC	
>	HomeAssistant	192.168.1.122	Deqing Dusu...	30:AE:7B:EB:40:8B	
	DESKTOP-VDN...	192.168.1.123	Intel Corporate	9C:FC:E8:4E:B1:8C	

Esta IP permite establecer conexión remota mediante SSH para continuar la configuración.

5. Configuración Inicial del Sistema

Acceso SSH:

ssh root@IP_MAQUINA

```
C:\> Administrador: C:\WINDOWS\system32\cmd.exe - ssh root@192.168.1.122

Microsoft Windows [Versión 10.0.26100.4652]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Administrator>ssh root@192.168.1.122
root@192.168.1.122's password:
Linux HomeAssistant 5.10.198 #93 SMP Thu Mar 6 15:39:44 CST 2025 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Feb  2 15:29:50 2026 from 192.168.1.117
root@HomeAssistant:~#
```

Actualizar sistema:

apt update

apt install nano

Se recomienda cambiar credenciales por defecto y configurar swap en entornos con recursos limitados para mejorar estabilidad.

6. Configuración Gateway LoRaWAN

Se crea un servicio systemd para el Packet Forwarder (lora_pkt_fwd) asegurando arranque automático y reinicio ante fallos.

Verificación mediante:

- systemctl status lora_pkt_fwd.service

- journalctl -u lora_pkt_fwd.service

```
root@HomeAssistant:~# systemctl status lora_pkt_fwd.service
● lora_pkt_fwd.service - LoRa Packet Forwarder
   Loaded: loaded (/etc/systemd/system/lora_pkt_fwd.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-02-11 16:58:31 CST; 4min 0s ago
     Main PID: 18524 (lora_pkt_fwd)
        Tasks: 4 (limit: 2300)
       Memory: 1.2M
      CGroup: /system.slice/lora_pkt_fwd.service
             └─18524 /root/packet_forwarder/lora_pkt_fwd -c /root/packet_forwarder/global_conf.json

Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: INFO: [down] PULL_ACK received in 1 ms
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: INFO: [down] PULL_ACK received in 1 ms
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: ##### 2026-02-11 09:01:05 GMT #####
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: ### [UPSTREAM] ###
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: # RF packets received by concentrator: 0
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: # CRC_OK: 0.00%, CRC_FAIL: 0.00%, NO_CRC: 0.00%
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[18524]: # RF packets forwarded: 0 (
Feb 11 17:01:05 HomeAssistant lora_pkt_fwd[22132]: Failed to connect to ubus
Feb 11 17:01:35 HomeAssistant lora_pkt_fwd[22793]: Failed to connect to ubus
Feb 11 17:02:05 HomeAssistant lora_pkt_fwd[23444]: Failed to connect to ubus
root@HomeAssistant:~#
```

El estado “active (running)” confirma que el gateway LoRaWAN está operativo y enviando paquetes al servidor.

7. Configuración MQTT (Mosquitto)

Configuración del broker MQTT para integración con ChirpStack y Home Assistant.

⚠ En entornos productivos se recomienda deshabilitar acceso anónimo y habilitar autenticación segura.

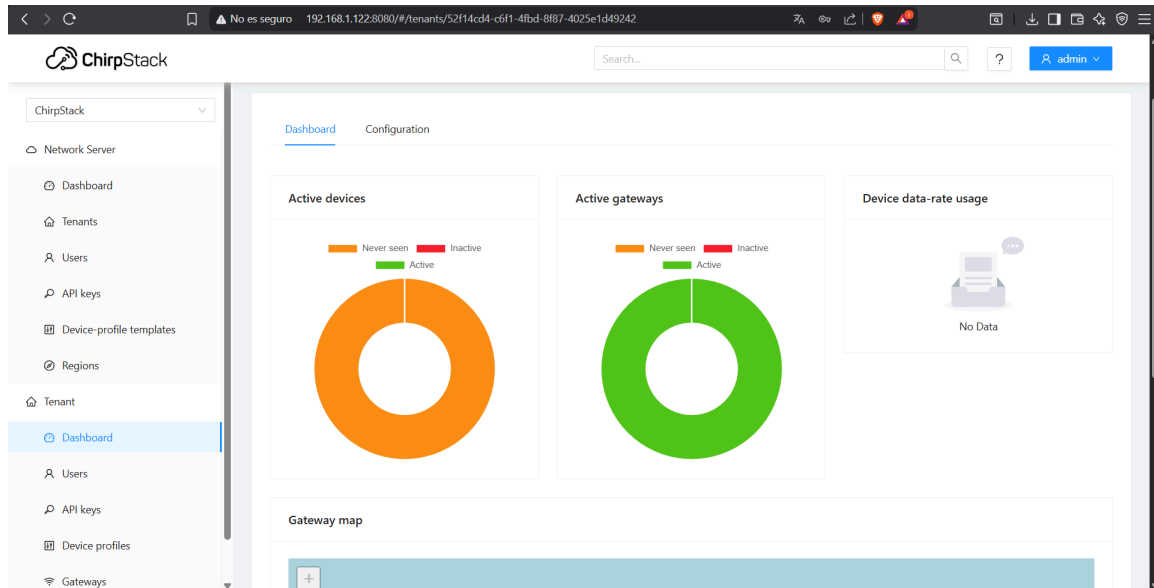
8. Instalación y Configuración de ChirpStack

Despliegue mediante Docker Compose.

Configuración de Gateway, Applications y Device Profiles.

Acceso web a través del puerto 8080.

El estado “Active” indica que el gateway está comunicándose correctamente con el servidor de red.



En ChirpStack, una **Application** actúa como contenedor lógico que agrupa dispositivos LoRaWAN que pertenecen a un mismo proyecto o caso de uso.

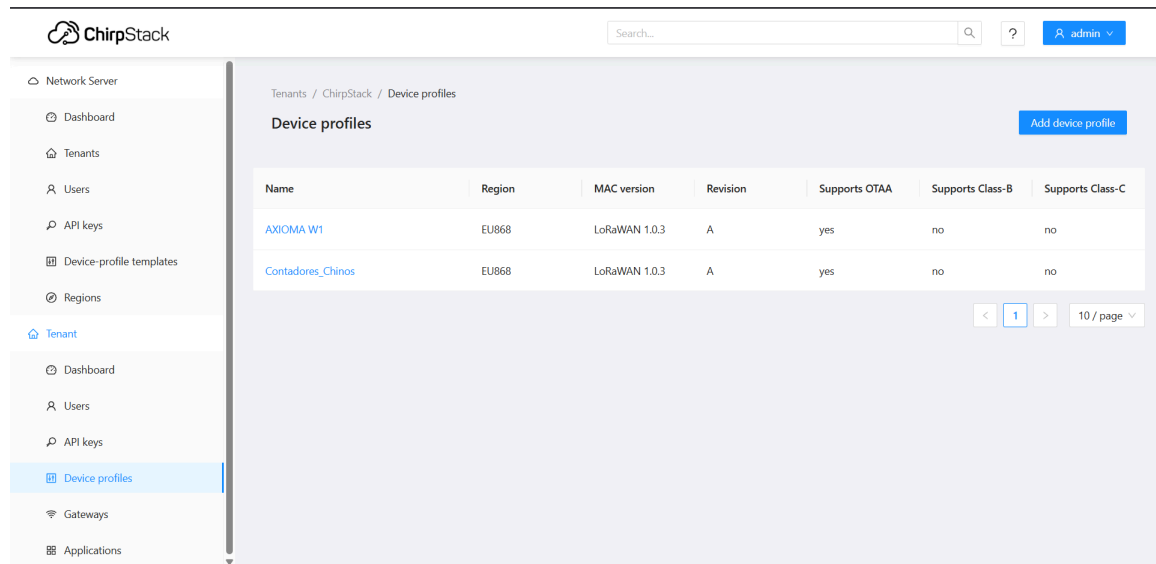
En este caso, se creó la aplicación **AQUADOMUS** para centralizar la gestión de los contadores de agua desplegados.

The screenshot shows the ChirpStack web interface for the 'AQUADOMUS' application. The left sidebar contains navigation links for Network Server, Tenant, and various configuration options. The main content area displays the application configuration page, including a table of devices and a 'Delete application' button.

☐	Last seen	DevEUI	Name	Device profile	Battery
☐	Never	2530152510300002	AXIOMA W1	AXIOMA W1	
☐	Never	2530152510300001	Contadores_Chinos	Contadores_Chinos	

En LoRaWAN, el *Device Profile* define las características técnicas del dispositivo, incluyendo:

- Clase del dispositivo (A, B o C)
- Región (EU868 en este caso)
- Versión LoRaWAN
- Configuración de payload y decoders



The screenshot displays the ChirpStack web interface. The sidebar on the left contains navigation links: Network Server, Dashboard, Tenants, Users, API keys, Device-profile templates, Regions, Tenant, Dashboard, Users, API keys, Device profiles (highlighted), Gateways, and Applications. The main content area is titled 'Tenants / ChirpStack / Device profiles' and shows a table of device profiles. The table has columns: Name, Region, MAC version, Revision, Supports OTAA, Supports Class-B, and Supports Class-C. Two profiles are listed: 'AXIOMA W1' and 'Contadores_Chinos', both for EU868 region and LoRaWAN 1.0.3 version. A 'Add device profile' button is in the top right corner. The bottom right of the table shows pagination: '< 1 >' and '10 / page'.

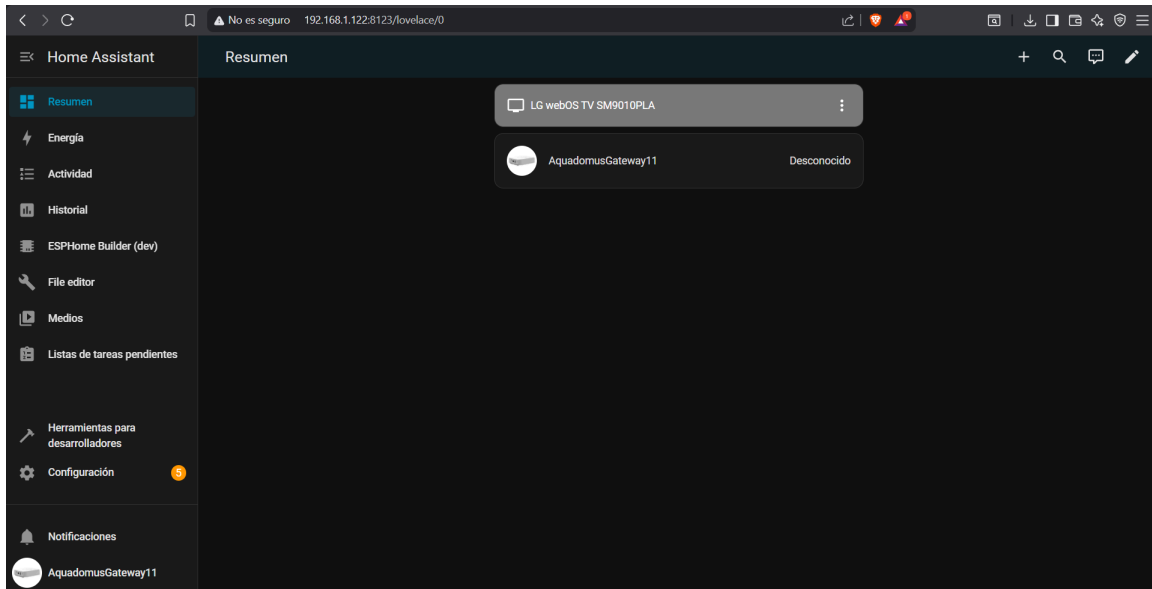
Name	Region	MAC version	Revision	Supports OTAA	Supports Class-B	Supports Class-C
AXIOMA W1	EU868	LoRaWAN 1.0.3	A	yes	no	no
Contadores_Chinos	EU868	LoRaWAN 1.0.3	A	yes	no	no

El Device Profile actúa como plantilla técnica que se asigna a cada dispositivo al registrarlo en la aplicación, asegurando coherencia en la configuración de red.

9. Integración con Home Assistant

Integración mediante MQTT para visualización de consumo, estado de batería y alarmas.

Se recomienda utilizar credenciales únicas por dispositivo para reforzar seguridad.

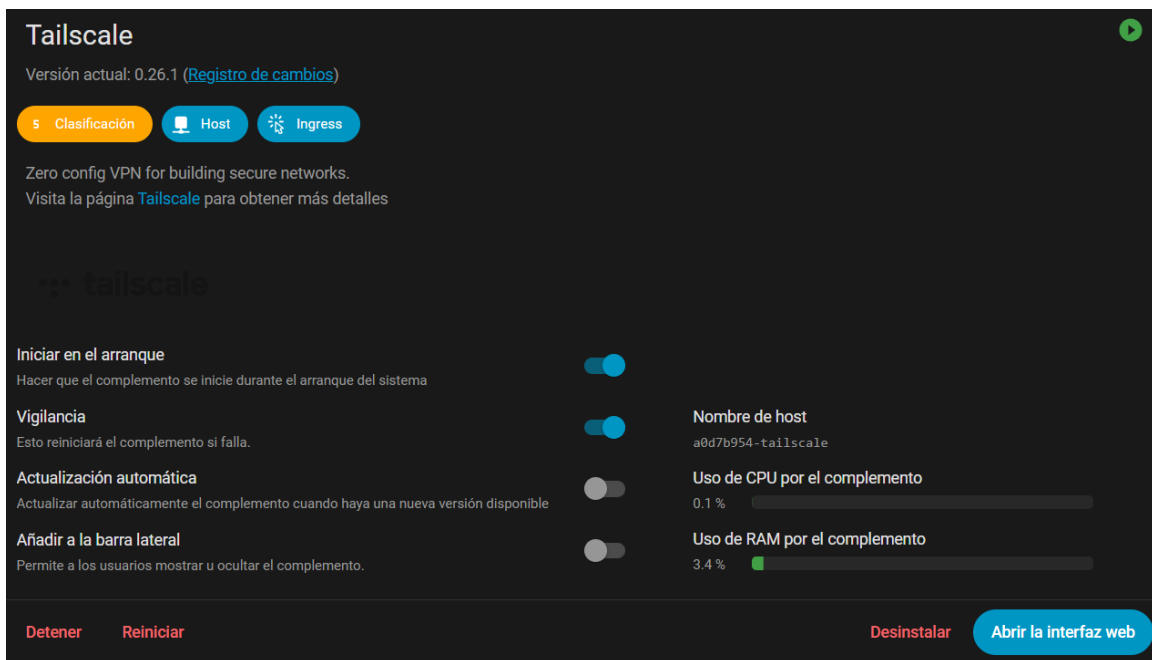


Esto permite monitorizar en tiempo real el volumen consumido y el estado del dispositivo.

Se creó un usuario específico y se verificó la correcta recepción de entidades desde ChirpStack.

10. Acceso Remoto Seguro – Tailscale

Implementación de red privada basada en WireGuard que permite acceso remoto cifrado sin necesidad de apertura de puertos en el router.



Permite acceder a Home Assistant y a la máquina DUSUN desde cualquier ubicación de forma cifrada.

11. Buenas Prácticas y Mantenimiento

- Monitorización periódica de logs
- Actualizaciones regulares del sistema
- Copias de seguridad de Docker
- Gestión segura de credenciales
- Validación de conectividad LoRa

12. Conclusión

La solución implementada permite desplegar una infraestructura IoT LoRaWAN robusta, escalable y segura, adecuada para proyectos de monitorización de consumo de agua en entornos residenciales, industriales o municipales.

La arquitectura garantiza modularidad, mantenibilidad y capacidad de crecimiento futuro.

13. Anexo Técnico – Comandos de Configuración

Desde WINDOWS:

```
scp -r "C:\Users\Administrator\Desktop\Synergia  
Tech\chirpstack-packet_forwarder\chirpstack-docker" root@IPMAQUINA:~/
```

```
scp -r "C:\Users\Administrator\Desktop\Synergia  
Tech\chirpstack-packet_forwarder\packet_forwarder" root@IPMAQUINA:~/
```

```
ssh root@IPMAQUINA
```

```
sudo apt update
```

```
sudo apt install nano
```

```
sudo nano /etc/hosts
```

- Añadir el nombre del HomeAssistant en 127.0.0.1 y ::1

```
sudo falldate -l 1G /swapfile
```

```
sudo chmod 600 /swapfile
```

```
sudo mkswap /swapfile
```

```
sudo swapon /swapfile
```

```
sudo nano /etc/fstab
```

y añadir la línea: /swapfile swap swap defaults 0 0

```
sudo reboot
```

1.1 Instalador de DUSUN

Crear servicio packet forwarder:

Cómo crear un servicio systemd para el packet forwarder (lora_pkt_fwd)

Así podrás iniciar, parar y reiniciar el gateway fácilmente, y que arranque automáticamente al boot.

1. Crear archivo de servicio

Abre un editor y crea el archivo:

```
nano /etc/systemd/system/lora_pkt_fwd.service
```

2. Pega esta configuración mínima, ajusta rutas si fuera necesario:

[Unit]

Description=LoRa Packet Forwarder

After=network.target

[Service]

Type=simple

User=root

ExecStart=/root/packet_forwarder/lora_pkt_fwd -c
/root/packet_forwarder/global_conf.json

Restart=always

RestartSec=10

[Install]

WantedBy=multi-user.target

3. Guardar y salir (en nano: Ctrl+O, Enter, Ctrl+X)

4. **Recargar systemd para detectar el nuevo servicio**

```
sudo systemctl daemon-reload
```

5. **Habilitar para que arranque al iniciar el sistema**

```
sudo systemctl enable lora_pkt_fwd.service
```

7. **Iniciar el servicio**

```
sudo systemctl start lora_pkt_fwd.service
```

7. **Comprobar estado**

```
sudo systemctl status lora_pkt_fwd.service
```

```
journalctl -u lora_pkt_fwd.service -f
```

```
sudo journalctl -u lora_pkt_fwd.service
```

1.2 **SERVICIO MOSQUITTO**

```
sudo nano /etc/mosquitto/conf.d/external_access.conf
```

copiar estas dos líneas:

```
listener 1883 0.0.0.0
```

```
allow_anonymous true
```

```
sudo systemctl restart mosquitto
```

packet forwarder

```
cd packet_forwarder/
```

```
cp reset_lgw.sh /usr/bin/
```

```
chmod +x /usr/bin/reset_lgw.sh
```

```
chmod +x lora_pkt_fwd
```

```
sudo nano global_conf.json
```

(cambiar "gateway_ID" y "server_address")

INFO: concentrator EUI: 0016c001f102192a

2. CONFIGURAR CHIRPSTACK

cd

cd chirpstack-docker/

sudo nano docker-compose.yml (Modificar IP Mosquitto)

sudo chown -R \$USER:\$USER .

chmod -R 755 configuration

cd configuration/chirpstack-gateway-bridge/

sudo nano chirpstack-gateway-bridge.toml (cambiar ip)

docker compose up -d

Para acceder a ChirpStack introduce en el navegador IPMAQUINA:8080

Configurar Gateway (ID GATEWAY)

Configurar Aplicación (AQUADOMUS)

Configurar Device Profiles (AXIOMA W1 y CONTADORES_CHINOS)

2.1 Javascript AXIOMA W1:

```
function decodeUplink(input) {
```

```
    const bytes = input.bytes;
```

```
    const fPort = input.fPort;
```

```
    function readUInt32LE(b, i) {
```

```
        return (b[i]) | (b[i + 1] << 8) | (b[i + 2] << 16) | (b[i + 3] << 24);
```

```
    }
```

```
    function readUInt16LE(b, i) {
```

```
        return (b[i]) | (b[i + 1] << 8);
```

```
    }
```

```

function unixToISO(ts) {
    return new Date(ts * 1000).toISOString();
}

/* =====
PORT 100 – MEASUREMENT DATA
===== */
if (fPort === 100) {
    let i = 0;

    const currentTimestamp = readUInt32LE(bytes, i); i += 4;
    const status = bytes[i]; i += 1;
    const currentVolumeLiters = readUInt32LE(bytes, i); i += 4;

    const logTimestamp = readUInt32LE(bytes, i); i += 4;
    const logVolumeLiters = readUInt32LE(bytes, i); i += 4;

    let deltas = [];
    for (let n = 1; n <= 15 && (i + 1) < bytes.length; n++) {
        const delta = readUInt16LE(bytes, i);
        i += 2;
        deltas.push({
            index: n,
            delta_liters: delta,
            delta_m3: delta / 1000
        });
    }
}

```

```
}
```

```
// Cálculo de volúmenes históricos acumulados
```

```
let historical = [];
```

```
let accVolume = logVolumeLiters;
```

```
let ts = logTimestamp;
```

```
deltas.forEach((d, idx) => {
```

```
  accVolume += d.delta_liters;
```

```
  ts += 3600; // por defecto 1h según documento
```

```
  historical.push({
```

```
    index: idx + 1,
```

```
    timestamp: unixToISO(ts),
```

```
    volume_liters: accVolume,
```

```
    volume_m3: accVolume / 1000
```

```
  });
```

```
});
```

```
return {
```

```
  data: {
```

```
    type: "axioma_w1_extended",
```

```
    timestamp: unixToISO(currentTimestamp),
```

```
    status_raw: status,
```

```
    status: decodeStatus(status),
```

```
    volume_liters: currentVolumeLiters,
```

```
    volume_m3: currentVolumeLiters / 1000,
```

```

log: {
    timestamp: unixToISO(logTimestamp),
    volume_liters: logVolumeLiters,
    volume_m3: logVolumeLiters / 1000
},
deltas: deltas,
historical: historical
}
};
}

```

```

/* =====
PORT 103 - ALARMS
===== */
if (fPort === 103) {
    const ts = readUInt32LE(bytes, 0);
    const status = bytes[4];

    return {
        data: {
            type: "axioma_w1_alarm",
            timestamp: unixToISO(ts),
            status_raw: status,
            status: decodeStatus(status)
        }
    };
}

```



```

    }

    return {
        errors: ["Unsupported FPort"]
    };
}

/* =====

STATUS DECODER

===== */
function decodeStatus(status) {
    return {
        low_battery: (status & 0x04) !== 0,
        permanent_error: (status & 0x08) !== 0,
        temporary_error: (status & 0x10) !== 0,
        leakage: (status & 0x20) !== 0,
        backflow: (status & 0x60) === 0x60,
        burst: (status & 0xA0) === 0xA0,
        freeze: (status & 0x80) !== 0,
        raw: "0x" + status.toString(16).padStart(2, "0")
    };
}

```

2.2 Javascript CONTADORES CHINOS:

/**

* Decoder corregido con filtrado de paquetes MAC y Device Status

```
*/  
  
function decodeUplink(input) {  
    var bytes = input.bytes;  
    var fPort = input.fPort;  
  
    // FILTRO 1: Ignorar paquetes MAC Command (fPort 0)  
    if (fPort === 0) {  
        return {  
            data: {},  
            warnings: ["MAC Command - no user data"]  
        };  
    }  
  
    // FILTRO 2: Ignorar paquetes vacíos o de Device Status  
    if (!bytes || bytes.length === 0) {  
        return {  
            data: {},  
            warnings: ["Empty payload - Device Status message"]  
        };  
    }  
  
    // FILTRO 3: Validar que el fPort sea el esperado (10 en tu caso)  
    if (fPort !== 10) {  
        return {  
            data: {},  
            warnings: ["Unexpected fPort: " + fPort]  
        };  
    }  
}
```

```

    };
}

var data = {};

// 1. Sincronización para ignorar el relleno 'fefe'
var start = bytes.indexOf(0x68);

// Validación de seguridad: Si no hay 0x68 o el paquete es muy corto
if (start === -1 || bytes.length < (start + 27)) {
    return {
        data: {},
        errors: ["Payload invalido o incompleto"]
    };
}

// --- VARIABLES PARA HOME ASSISTANT ---

// 2. Meter Address (7 bytes según V1.2)
data.meterAddress = "";
for (var i = start + 3; i < start + 10; i++) {
    data.meterAddress += bytes[i].toString(16).padStart(2, '0');
}

// 3. Estado de Válvula
var valveStatus = bytes[start + 13];

```

```
data.valve = {  
    on: (valveStatus & 0x01) !== 0,  
    off: (valveStatus & 0x01) === 0,  
    abnormal: (valveStatus & 0x80) !== 0  
};
```

```
// 4. Alertas
```

```
var alertByte = bytes[start + 14];  
data.alerts = {  
    leakage: (alertByte & 0x01) !== 0,  
    unguarded: (alertByte & 0x02) !== 0,  
    abnormalWaterUse: (alertByte & 0x04) !== 0  
};
```

```
// 5. Lectura de Flujo
```

```
function readFloatLE(b, s) {  
    var buf = new ArrayBuffer(4);  
    var view = new DataView(buf);  
    for (var i = 0; i < 4; i++) view.setUint8(i, b[s + i]);  
    return view.getFloat32(0, true);  
}  
  
function readInt32LE(b, s) {  
    return (b[s + 3] << 24) | (b[s + 2] << 16) | (b[s + 1] << 8) | b[s];  
}
```

```
// Cálculo corregido (Offsets alineados tras detectar el 0x68)
```

```

var integerPart = readInt32LE(bytes, start + 20);
var fractionalPart = readFloatLE(bytes, start + 24);

data.totalFlow = (integerPart + fractionalPart).toFixed(3);

// Variables adicionales
data.batteryVoltage = readFloatLE(bytes, start + 16).toFixed(2);

// El flujo residual suele estar al final, justo antes del Checksum (1 byte) y End Code (1
byte)
if (bytes.length >= 6) {
    data.residualFlow = readFloatLE(bytes, bytes.length - 6).toFixed(2);
}

return { data: data };
}

/**
 * Encoder para LoRaWAN Flowmeter V1.2
 */
function encodeDownlink(input) {
    var data = input.data;
    var bytes = [];

    bytes.push(0x68);
    bytes.push(0x70);

```

```
var addrStr = data.meterAddress || "0000000000000000";

if (addrStr.length > 14) addrStr = addrStr.substring(addrStr.length - 14);

for (var i = 0; i < 14; i += 2) {
    bytes.push(parseInt(addrStr.substr(i, 2), 16));
}

var ctrl0 = 0x00;

if (data.command_type === "open" || data.command_type === "close") ctrl0 |= 0x01;

if (data.recharge !== undefined) ctrl0 |= 0x08;

bytes.push(ctrl0);

var lengthIndex = bytes.length;

bytes.push(0x00);

var now = new Date();

bytes.push(parseInt(now.getFullYear().toString().slice(-2), 16));

bytes.push(parseInt((now.getMonth() + 1).toString().padStart(2, '0'), 16));

bytes.push(parseInt(now.getDate().toString().padStart(2, '0'), 16));

bytes.push(parseInt(now.getHours().toString().padStart(2, '0'), 16));

bytes.push(parseInt(now.getMinutes().toString().padStart(2, '0'), 16));

bytes.push(parseInt(now.getSeconds().toString().padStart(2, '0'), 16));

if (data.command_type === "open") {
    bytes.push(0x01);
}
```

```

    } else if (data.command_type === "close") {
        bytes.push(0x00);
    }

    if (data.recharge !== undefined) {
        bytes.push(data.recharge.sequence & 0xFF);
        var buffer = new ArrayBuffer(4);
        var view = new DataView(buffer);
        view.setFloat32(0, data.recharge.volume, true);
        for (var j = 0; j < 4; j++) bytes.push(view.getUint8(j));
    }

    bytes[lengthIndex] = bytes.length - lengthIndex - 1;

    var checksum = 0;
    for (var k = 0; k < bytes.length; k++) checksum += bytes[k];
    bytes.push(checksum & 0xFF);
    bytes.push(0x16);

    return {
        bytes: bytes,
        fPort: 10
    };
}

```

3. COMANDO CLAVE DESPUES DE HABER PUESTO LAS IPs:

```
- cd chirpstack-docker/
```

- Comando: docker-compose up -d --force-recreate chirpstack-gateway-bridge-eu868.
- Nota: Esto elimina el rastro de IPs antiguas.

4. CONFIGURAR WIFI

1. Instalar herramientas necesarias

```
apt update
```

```
apt install -y wpasupplicant rfkill iw
```

2. Activar las interfaces

```
ip link set wlan0 up
```

```
ip link set wlan1 up
```

3. Activar el radio Wi-Fi en NetworkManager

```
nmcli radio wifi on
```

4. Habilitar y arrancar wpa_supplicant

```
systemctl enable --now wpa_supplicant
```

5. Reiniciar NetworkManager

```
systemctl restart NetworkManager
```

6. Verificar que las interfaces están disponibles

```
nmcli dev status
```

7. Escanear redes (opcional, para verificar)

```
nmcli dev wifi list
```


5. CONFIGURACION HOME ASSISTANT:

Crear el usuario de Home Assistant (Para acceder a Home Assistant en el navegador introduce IPMAQUINA:8123)

Usr: aquadomusgatewayX

Pwd: aquadomusgatewayX

Instalar Tailscale en setting – Add-ons – y conectar con almomir

Cambiar el nombre de Tailscale al nombre de home Assistant (aquadomusgatewayX).